1 Problem Formulation: Bundle Optimization in a Rental Marketplace

We consider a rental marketplace in which a user requests a bundle of rental items for a given time interval, location, and maximum budget. Unlike a standard marketplace search, the goal is not to return individual products independently. Instead, the system constructs a complete bundle by selecting one suitable item for each requested requirement.

2 Input Definition

Let:

$$S = \{1, 2, \ldots, n\}$$

be the set of required item slots.

For each slot $s \in S$, let:

$$I_s = \{i_1, i_2, \ldots, i_k\}$$

be the set of candidate rental items that can satisfy requirement s.

The global candidate set is:

$$I = \bigcup_{s \in S} I_s$$

Each item $i \in I$ has:

- p_i – rental price
- d_i – distance from the user
- r_i – lender reliability score
- c_i – item condition score
- a_i – availability score
- $L(i)$ – lender / pickup owner

3 Decision Variables

$$x_i = \begin{cases} 1 & \text{if item } i \text{ is selected} \\ 0 & \text{otherwise} \end{cases}$$

4 Hard Constraints

4.1 Assignment Constraint

Exactly one item must be selected for each requested slot:

$$\forall s \in S : \sum_{i \in I_s} x_i = 1$$

4.2 Budget Constraint

$$\sum_{i \in I} p_i x_i \leq B$$

4.3 Availability Constraint

An unavailable item cannot be selected:

$$x_i \leq a_i$$

4.4 Minimum Condition Constraint

$$c_i \geq c_{\min}$$

5 Bundle-Level Metrics

5.1 Total Price

$$P(x) = \sum_{i \in I} p_i x_i$$

The price score is budget-relative:

$$M_{\text{price}}(x) = \begin{cases} 0 & \text{if } 10\left(1 - \frac{P(x)}{B}\right) < 0 \\ 10\left(1 - \frac{P(x)}{B}\right) & 0 \leq 10\left(1 - \frac{P(x)}{B}\right) \leq 10 \\ 10 & \text{if } 10\left(1 - \frac{P(x)}{B}\right) > 10 \end{cases}$$

5.2 Distance

Distances are computed from geocoded coordinates using the Haversine formula.

Let:

$$\bar{d}(x) = \frac{1}{n} \sum_{i \in I} d_i x_i$$

$$d_{\max}(x) = \max_{i : x_i = 1} d_i$$

The distance metric combines average distance and the farthest selected item:

$$D(x) = \eta \cdot \bar{d}(x) + (1 - \eta) \cdot d_{\max}(x)$$

The normalized distance score is:

$$M_{\text{distance}}(x) = 10 \cdot e^{-D(x)/30}$$

A separate max-distance penalty is also applied:

$$Penalty_{\max d}(x) = \gamma \cdot \left(1 - e^{-d_{\max}(x)/30}\right)$$

5.3 Reliability

The lender reliability score is based on rating, transaction history, cancellations, complaints, response score, and verification level.

A Bayesian adjusted rating is used:

$$sampleWeight = \min\left(1, \frac{completedTransactions}{30}\right)$$

$$bayesianRating = rating \cdot sampleWeight + 3.7 \cdot (1 - sampleWeight)$$

Then:

$$ratingScore = 2 \cdot bayesianRating$$

New or unproven lenders receive an uncertainty penalty:

$$newPenalty = \begin{cases} 1.5 & completedTransactions < 2 \\ 1.0 & completedTransactions < 5 \\ 0 & \text{otherwise} \end{cases}$$

The reliability score is clamped to:

$$r_i \in [0, 10]$$

Bundle reliability is computed as:

$$M_{\text{reliability}}(x) = \frac{1}{n} \sum_{i \in I} r_i x_i$$

5.4 Product Condition

Each item condition is mapped to a numeric score:

$$NEW = 10, \quad LIKE\_NEW = 9, \quad GOOD = 7.5, \quad FAIR = 5, \quad HEAVY\_USE = 2$$

A pure average can hide one weak item. Therefore, the bundle condition score is bottleneck-aware:

$$\bar{c}(x) = \frac{1}{n} \sum_{i \in I} c_i x_i$$

$$c_{\min}(x) = \min_{i : x_i = 1} c_i$$

$$M_{\text{condition}}(x) = 0.6 \cdot \bar{c}(x) + 0.4 \cdot c_{\min}(x)$$

5.5 Availability

Availability is first treated as a hard constraint. Items blocked, booked, under maintenance, or outside their rental-day limits are removed.

The bundle availability score is conservative:

$$M_{\text{availability}}(x) = \min_{i:x_i=1} a_i$$

5.6 Pickup Complexity

Let $Q(x)$ be the set of unique pickup points in the bundle.

$$Q(x) = \{L(i) \mid x_i = 1\}$$

Pickup complexity includes both the number of pickup points and the spatial distance between them:

$$PickupCost(x) = \frac{1}{25} \sum_{q_a, q_b \in Q(x)} dist(q_a, q_b) + \max(0, |Q(x)| - 1)$$

The pickup penalty is:

$$Penalty_{\text{pickup}}(x) = \beta \cdot PickupCost(x)$$

6 Metric Normalization

All bundle metrics are normalized to:

$$M_j(x) \in [0, 10]$$

where higher is better.

The five core metrics are:

$$M(x) = (M_{\text{price}}, M_{\text{distance}}, M_{\text{reliability}}, M_{\text{condition}}, M_{\text{availability}})$$

7 Weighted Utility

The user's preferences are represented by weights:

$$w_1, w_2, \ldots, w_m$$

where:

$$\sum_{j=1}^{m} w_j = 1$$

The weighted utility is:

$$U(x) = \sum_{j=1}^{m} w_j M_j(x)$$

8 Variance Penalty

The mean metric value is:

$$\bar{M}(x) = \frac{1}{m} \sum_{j=1}^{m} M_j(x)$$

The variance is:

$$Var(M(x)) = \frac{1}{m} \sum_{j=1}^{m} \left(M_j(x) - \bar{M}(x)\right)^2$$

The variance penalty is:

$$Penalty_{\text{variance}}(x) = \lambda \cdot Var(M(x))$$

9 Bottleneck Bonus

The bottleneck score is:

$$B_n(x) = \min_j M_j(x)$$

The bottleneck bonus is:

$$Bonus_{\text{bottleneck}}(x) = \alpha \cdot B_n(x)$$

10 Low Score Penalty

To strongly punish weak dimensions, the system adds an explicit low-score penalty.
For each metric j:

$$Penalty_j(x) = \rho_j \cdot \max(0, \tau_j - M_j(x))$$

where:

- τ_j is the minimum acceptable threshold for metric j
- ρ_j is the penalty weight for metric j

The total low-score penalty is:

$$Penalty_{\text{low}}(x) = \sum_{j=1}^{m} \rho_j \cdot \max(0, \tau_j - M_j(x))$$

Default thresholds:

$$\tau_{\text{price}} = 4, \quad \tau_{\text{distance}} = 4, \quad \tau_{\text{reliability}} = 6.5, \quad \tau_{\text{condition}} = 6.5, \quad \tau_{\text{availability}} = 7$$

This means that low product condition, weak lender reliability, poor availability, high distance, or poor price fit directly reduce the final score.

11 Final Objective Function

The raw final score is:

$$\begin{aligned} Score_{\text{raw}}(x) = U(x) - Penalty_{\text{variance}}(x) + Bonus_{\text{bottleneck}}(x) \quad (1) \\ - Penalty_{\text{pickup}}(x) - Penalty_{\max d}(x) - Penalty_{\text{low}}(x) \quad (2) \end{aligned}$$

The displayed score is clamped to the range $[0, 10]$:

$$Score(x) = \begin{cases} 0 & \text{if } Score_{\text{raw}}(x) < 0 \\ Score_{\text{raw}}(x) & 0 \le Score_{\text{raw}}(x) \le 10 \\ 10 & \text{if } Score_{\text{raw}}(x) > 10 \end{cases}$$

The optimization problem is:

$$\max_x Score(x)$$

subject to:

$$\forall s \in S : \sum_{i \in I_s} x_i = 1$$

$$\sum_{i \in I} p_i x_i \le B$$

$$x_i \le a_i$$

$$x_i \in \{0, 1\}$$

12 Algorithmic Solution Approach

Solving the problem exactly is computationally expensive. If each requested slot has k candidates and the user requests n items, the naive search space is:

$$k^n$$

For example:

$$30^5 = 24,300,000$$

possible bundles.
Therefore, the system uses a heuristic search strategy.

12.1 Step 1: Constraint-Based Filtering

The system first removes infeasible items:

- unavailable items
- inactive listings
- items below minimum condition
- items outside price constraints
- items outside distance constraints
- items violating minimum or maximum rental days

This reduces:

$$I_s \to \tilde{I}_s$$

12.2 Step 2: Candidate Pruning

For each slot s, the system keeps only the top-k candidates:

$$I'_s \subseteq \tilde{I}_s$$

$$|I'_s| \leq k$$

The preliminary score considers price, distance, reliability, condition, and availability.

12.3 Step 3: Beam Search

Let B_t be the set of partial bundles after processing t slots.

At step $t+1$, every partial bundle is extended with candidates from the next slot. Then only the top w partial bundles are retained:

$$|B_t| \leq w$$

This changes the approximate complexity from:

$$O(k^n)$$

to:

$$O(n \cdot w \cdot k)$$

12.4 Step 4: Final Scoring

Each complete bundle is scored using:

$$Score(x) = \begin{cases} 0 & \text{if } Score_{\text{raw}}(x) < 0 \\ Score_{\text{raw}}(x) & 0 \leq Score_{\text{raw}}(x) \leq 10 \\ 10 & \text{if } Score_{\text{raw}}(x) > 10 \end{cases} \tag{3}$$

where (4)

$$Score_{\text{raw}}(x) = U(x) - Penalty_{\text{variance}}(x) + Bonus_{\text{bottleneck}}(x) \tag{5}$$

$$- Penalty_{\text{pickup}}(x) - Penalty_{\max d}(x) - Penalty_{\text{low}}(x) \tag{6}$$

12.5 Step 5: Explanation Generation

The system explains why a bundle was recommended or penalized.

Examples:

- the bundle is under budget
- the pickup route is short
- all items are available
- one item has relatively low condition
- one lender has relatively low reliability
- one pickup point is far away

13 System Output

The system returns a ranked list:

$$x^{(1)}, x^{(2)}, \ldots, x^{(q)}$$

where:

$$Score(x^{(1)}) \geq Score(x^{(2)}) \geq \cdots \geq Score(x^{(q)})$$

The top bundle is presented as the recommended bundle.

14 Conclusion

The rental bundle selection problem is formulated as a constrained multi-objective combinatorial optimization problem.

The system selects one item per requested slot while respecting budget, availability, location, condition, and inventory constraints. It then ranks feasible bundles using a score that combines price, distance, reliability, condition, availability, pickup complexity, variance, bottleneck awareness, max-distance penalty, and low-score penalties.

The key contribution is that the system does not simply maximize an average score. It actively penalizes weak dimensions, poor product condition, low lender reliability, long pickup distances, and logistical complexity. This makes the recommended bundle not only good on average, but also balanced, practical, and robust.

Implementation Note The mathematical formulation describes the conceptual scoring model. In the actual system, additional engineering considerations such as fallback values, data availability, and performance optimizations slightly modify the exact computations.

Model vs. Implementation. The mathematical formulation above presents a clean and structured optimization model. However, the actual system implementation introduces several deviations motivated by practical engineering constraints.

First, while the model assumes continuous and well-defined metric functions $M_j(x)$, the implementation relies on discrete data, fallback values, and partial information. For example, availability is treated primarily as a hard constraint (items are filtered out) rather than a smooth scoring function, and missing data fields may be replaced with default values.

Second, some scoring components differ in their exact formulation. The distance metric in the model is expressed as a weighted combination of average and maximum distance, whereas in practice it is normalized using heuristic functions and combined with an additional explicit maximum-distance penalty. Similarly, pickup complexity is modeled abstractly as the number of unique lenders, but the implementation may incorporate spatial dispersion between pickup points.

Third, the reliability model is simplified mathematically, but the implementation uses a more detailed aggregation including Bayesian smoothing, transaction counts, verification

levels, and penalties for new or unproven lenders. These additions ensure robustness against sparse or biased data.

Fourth, although the objective function is written in a compact analytical form, the system does not solve it exactly. The combinatorial nature of the problem leads to an exponential search space, so the implementation relies on heuristic search techniques (filtering, candidate pruning, and beam search). As a result, the returned solution is an approximation of the optimal bundle rather than a guaranteed global optimum.

Finally, the implementation includes additional safeguards not explicitly modeled, such as score clamping to the range $[0, 10]$, explicit low-score penalties for weak dimensions, and defensive handling of edge cases. These adjustments improve stability, interpretability, and user experience.

Overall, the mathematical model captures the conceptual structure of the optimization problem, while the implementation adapts it to real-world data, performance constraints, and system reliability requirements.

Key Insight. The strength of the system lies not in solving the exact mathematical formulation, but in approximating it effectively under real-world constraints.